**Lumi Reading · Security & Privacy Evidence**

EV13

# Secure Development Lifecycle

Lumi Education Pty Ltd · ABN 45 700 349 015 · 24 July 2026

DRAFT

**ST4S item:** EV13 (also evidences Q5 — security testing to a framework) **Version:** 0.1 DRAFT · **Date:** 2026-07-24 **Status:** Draft for review — not yet signed

## 1. Purpose and scope

This document describes Lumi Reading's actual secure development lifecycle: how a change moves from design, through peer review and automated CI gates, to a gated deployment, and how it is rolled back. Every control described here is implemented in the repository and cited by file path. Where a control is partial or pending, it is called out explicitly (see §11).

Scope: the Lumi monorepo (`lumi-ninc-au`, `australia-southeast1`) — the Flutter app (`lib/**`), Cloud Functions (`functions/src/**`), the privileged core (`packages/server-ops/**`), the two Next.js portals (`school-admin-web/`, `admin/`), the marketing site (`marketing-site/`), and the security rules (`firestore.rules`, `storage.rules`).

## 2. Lifecycle overview

```
design/review  →  branch + PR  →  automated CI gates  →  merge  →  gated deploy  →  monitor / rollb
  (§3, §4)         (§3)              (§5, §6)           (§3)        (§7, §8)         (§9, §10)
```

Two principles govern the flow:

- **Authorization is server-defined, tested as code.** The security boundary is the Firestore/Storage rules plus server-owned role fields, so rules changes are treated as code and gated by an emulator test suite (§6).
- **Deploys are deliberate.** Only the admin portal auto-deploys from `main` (`.github/workflows/admin-deploy.yml`). Cloud Functions, security rules, Firestore indexes, the school portal and the Flutter app are all deployed by an explicit, human-run `firebase deploy` / release step (§7).

## 3. Change management — branch → PR → squash-merge

Every non-trivial change is made on a prefixed branch (`feat/`, `fix/`, `refactor/`, `docs/`, `sec/`), pushed, opened as a pull request, and squash-merged into `main` with the branch auto-deleted. The squash commit message names the change (and, for security work, the ST4S item it addresses), which keeps `git log` a usable audit trail — every fix in the current security programme is a single named squash commit (e.g. rules fixes `0d31809` / PR #520, MFA-crypto `96b4cce` / PR #557, server-ops authz `6d5b795` / PR #561).

Pull requests are the review surface: a human reviewer plus the automated gates in §5–§6 must be satisfied before merge.

## 4. Design / release security-and-privacy review gate

A structured review gate applies to any change touching sensitive surfaces — auth/sessions/roles/tenancy, security rules or schemas, child/parent/staff personal information, audio/AI, analytics/logging, vendors/secrets/processing location, retention/deletion, or abuse-sensitive endpoints.

**Source:** `docs/privacy/RELEASE_PRIVACY_SECURITY_REVIEW.md`.

The gate has three checklists:

1. **Pull-request checklist** — state the personal data and principals affected; describe the expected school → class → child binding; require positive *and* negative (cross-tenant denial) tests; validate every accepted field/type/range and lock system fields; check subcollections and Storage separately; confirm queries are scope-bound; confirm no secret or Admin credential reaches a client/build/log; update the PIA/vendor register where relevant.
2. **Release evidence checklist** — Functions/unit tests pass; Firestore and Storage rules emulator suites pass; Flutter analysis and tests pass; portal typecheck/build pass where changed; production dependency audits carry no accepted critical/high (or a time-limited, owned exception); gitleaks clean; reviewed rules/config match the live deployment hashes.
3. **Recurring schedule** — weekly automated review, monthly IAM/keys review, quarterly vendor/PIA review, six-monthly breach tabletop (see the doc's schedule table, and §5/§9 below).

An unchecked mandatory item blocks release unless the named approver records a time-limited exception.

## 5. Continuous-integration gates

All workflows live in `.github/workflows/`. Triggers and enforcement:

### 5.1 `demo-readiness.yml` — the full regression gate (blocking)

- **Trigger:** every pull request, and push to `main`, that touches app, functions, portals, rules, or lockfiles (path-filtered); also manual dispatch.
- **Environment (pinned):** Node 22, Java 21 (Temurin), Flutter **3.44.6** (deliberately pinned — a floating `stable` channel previously turned `main` red on upstream Flutter releases; the pin is documented inline in the workflow).
- **Enforces (`pnpm test:demo-readiness` → `scripts/demo-readiness-local.sh`):** server-ops demo-plan/preflight/control tests, a demo-reseed emulator integration test, the Functions read-only-guard test, the **Firestore rules emulator suite** and the **Storage rules emulator suite** (§6), the full Flutter test suite plus `flutter analyze` on `lib/`, the school-portal security test + `tsc --noEmit`, and the admin-portal `tsc --noEmit`.
- A red run blocks the PR.

### 5.2 `admin-ci.yml` — admin/portal build gate (blocking)

- **Trigger:** PRs and push to `main` touching `admin/**`, `packages/**`, the audio-authority/cover-OCR sources, or lockfiles.
- **Enforces:** audio-authority parity, **feature-flag parity** (portal and Cloud Functions must resolve the same flag docs to the same state — a drift would silently mis-display a kill switch), TypeScript typecheck, lint, and a full Next build.

### 5.3 `security-review.yml` — weekly automated security review

- **Trigger:** scheduled `17 2 * * 1` (Mondays 02:17 UTC) + manual dispatch.

- **Enforces:** Functions unit tests (`npm run test:functions`), the **Firestore rules emulator suite** (`npm run test:rules`), a Functions production dependency audit (`npm audit --omit=dev --audit-level=high`), and a portal production dependency audit (`pnpm audit --prod --audit-level high`).

### 5.4 `secret-scan.yml` — secret scanning (gitleaks)

- **Trigger:** push to `main`, **every** pull request, and manual dispatch.
- **Enforces:** `gitleaks` v8.30.1 over the **complete git history** (`fetch-depth: 0`, `--redact=100`), honouring `.gitleaksignore`. The current history is gitleaks-clean.

### 5.5 `sca-osv.yml` — software composition analysis (osv-scanner)

- **Trigger:** monthly `17 3 1 * *` (1st of month, 03:17 UTC) + manual dispatch.
- **Enforces:** `osv-scanner` (Google action v2) recursively over every lockfile (npm + pub), uploading SARIF to the GitHub Security tab. **Report-only** (`continue-on-error`) while the findings register is triaged. The per-PR trigger is intentionally disabled pending action-ref verification (documented inline; re-enable once a green baseline confirms the config).

### 5.6 `sast-semgrep.yml` — static application security testing (semgrep)

- **Trigger:** monthly `23 3 1 * *` (1st of month, 03:23 UTC) + manual dispatch.
- **Enforces:** `semgrep scan` with community rulesets (`p/default`, `p/typescript`, `p/javascript`, `p/secrets`), uploading SARIF to the Security tab. **Report-only** while the baseline is triaged. Per-PR trigger disabled for the initial landing (documented inline). The current SAST baseline is one low finding (SAST-01, now fixed) with no secrets flagged.

### 5.7 `dependabot.yml` — dependency update automation

- **Source:** `.github/dependabot.yml`. **Weekly** updates for: npm in `/functions`, `/school-admin-web`, `/admin`, `/marketing-site` and root; `pub` at root (Flutter); and `github-actions` at root.

## 6. Rules-as-code testing (the primary authorization boundary)

Authorization is enforced by security rules, so the rules are tested like code against the Firebase Emulator Suite on synthetic data only:

- **`functions/test/firestore.rules.test.js`** (~4,100 lines) — the exhaustive Firestore rules matrix (positive and negative, cross-tenant, field-lock, query-scoping cases).
- **`functions/test/security_poc.rules.test.js`** — the security-assessment proof-of-concept + regression suite. Each test pairs a legitimate write that must keep working with the malicious write that must be denied, so a red test isolates to the vulnerability. It encodes the deployed fixes F-01, F-02, F-03 and a positive cross-tenant-isolation assurance test (a school-A teacher can neither read school-B data directly nor sweep it via an unconstrained collection-group query).
- **`functions/test/storage.rules.test.js`** — the Storage rules suite.

These run via `functions/package.json` scripts `test:rules` and `test:rules:storage` (both wrapped by `scripts/with-jdk21.sh`), and are wired into **both** the per-PR `demo-readiness` gate (§5.1) and the weekly `security-review` job (§5.3). A rules regression fails CI before it can merge.

## 7. Deploy gates

**Only the admin portal auto-deploys.** Everything else is a deliberate, human-run deploy — this is a known, documented property of the estate, not an oversight.

- **Admin portal (auto):** `.github/workflows/admin-deploy.yml` deploys `admin/` to Firebase Hosting on push to `main` (admin-affecting paths). It authenticates **keylessly** via GitHub OIDC / Workload Identity Federation (no JSON private key stored in GitHub), the WIF provider condition accepts only this repo's `main` branch, and the service-account binding is scoped to that principal. Before deploying it runs `infra/iam/audit-admin-build-identity.sh` to **refuse the deploy** if the build or runtime identity has drifted to a default account or gained project-data/secret/key access; after deploying it disables the redundant Firebase Frameworks auth bridge (`scripts/security/disable-admin-firebase-framework-auth-bridge.sh`). Deploys are serialized (`concurrency: admin-deploy-main`, no cancel-in-progress).
- **School portal (manual):** `FIREBASE_CLI_EXPERIMENTS=webframeworks firebase deploy --only hosting:school`. Gate before deploy is `tsc --noEmit` + Next build (there is no lint CI on the portal); never run `next build` against a live `next dev` server.
- **Cloud Functions (manual):** `firebase deploy --only functions`. The Functions predeploy hook runs ESLint + `tsc`, so non-lint-clean merged code blocks the deploy — a de-facto gate.
- **Security rules / indexes (manual):** `firebase deploy --only firestore:rules` / `firestore:indexes`. Note: an index deploy silently **deletes** remote indexes missing from `firestore.indexes.json`, so the remote set is dumped and merged before deploying.
- **Flutter app (manual):** built via `./scripts/flutter-build.sh <target>` and released through the app stores.

Post-deploy, the release evidence checklist (§4) requires confirming that the reviewed rules/config match the **live deployment hashes** and recording a production canary.

## 8. Rollback

- **Rules / functions / portals:** roll back by re-deploying the previous known- good revision (the squash-merge history gives a clean revert point; the admin portal keeps Cloud Run revisions). Undoing one's own uncommitted work is done by explicit edit, never by destructive `git reset` / `checkout` on a shared checkout.
- **Feature-level rollback without deploy:** kill switches and entitlement flags are Firestore-doc-driven (e.g. `platformConfig/incrementalAggregation`, notification caps, the AI-eval gates), so a feature can be turned off by flipping a document with no code deploy. Feature-flag parity is CI-enforced (§5.2) so a switch cannot display a state the feature is not in.
- **MFA enforcement:** the mandatory admin-TOTP portal gate has an explicit short-lived emergency rollback (`ADMIN_TOTP_ENFORCED=false`), documented in `docs/ADMIN_TOTP_MFA_RUNBOOK.md`.

## 9. Findings and remediation loop

Findings from the automated scanners and the self-managed assessment are tracked in a single register — `docs/security/FINDINGS_REGISTER.md` — one row per finding with severity, ST4S mapping, status, owner and decision. Fixed findings carry a regression test (rules fixes → `security_poc.rules.test.js`) so the same class cannot silently regress. As of 2026-07-24 every fixable code-security finding (F-01–F-04, F-07, F-09/A2, SAST-01) is fixed, regression-tested, merged and deployed; the remainder are accepted-by-design, launch-gated, or upstream-tracked (see the register and `VULNERABILITY_ASSESSMENT_REPORT_2026-07-24.md`).

## 10. Supporting evidence index

| Control | Evidence |
|---|---|
| Full regression gate | `.github/workflows/demo-readiness.yml`, `scripts/demo-readiness-local.sh` |
| Admin build/typecheck/flag-parity gate | `.github/workflows/admin-ci.yml` |
| Weekly automated security review | `.github/workflows/security-review.yml` |
| Secret scanning (history) | `.github/workflows/secret-scan.yml`, `.gitleaksignore` |
| SCA (osv-scanner) | `.github/workflows/sca-osv.yml` |
| SAST (semgrep) | `.github/workflows/sast-semgrep.yml` |
| Dependency updates | `.github/dependabot.yml` |
| Rules-as-code tests | `functions/test/firestore.rules.test.js`, `functions/test/security_poc.rules.test.js`, `functions/test/storage.rules.test.js` |
| Regression guardrails (fail-closed) | `scripts/check-storage-rules.sh`, `scripts/check-csv-exports.sh` — run in CI **and** `firebase.json` predeploy; each blocks a specific defect class from recurring (Storage-rules→Firestore reads; CSV/formula injection, F-10) |
| Release review gate | `docs/privacy/RELEASE_PRIVACY_SECURITY_REVIEW.md` |
| Admin auto-deploy + identity guard | `.github/workflows/admin-deploy.yml`, `infra/iam/audit-admin-build-identity.sh` |
| Findings register / assessment | `docs/security/FINDINGS_REGISTER.md`, `docs/security/VULNERABILITY_ASSESSMENT_REPORT_2026-07-24.md` |

## 11. Known gaps (for the reviewer)

- **Report-only scanners.** The osv-scanner and semgrep workflows currently run **monthly + on-demand only** and are report-only; their per-PR triggers are disabled pending action-ref / green-baseline verification. To claim these as a per-deploy blocking gate, the per-PR triggers must be re-enabled and made blocking. Until then, the blocking dependency gate is the weekly/PR `npm/pnpm audit` (production deps, high+) in §5.1/§5.3.
- **No lint CI on the school portal.** It is gated by `tsc` + Next build only (documented); a reviewer should confirm this is acceptable.
- **Manual deploys.** Because functions/rules/app are deployed by hand, the "reviewed config matches live hashes" release step (§4/§7) is the control that the deployed state matches what was reviewed — confirm this is being recorded for every release, not just at assessment time.
- **Approver identity.** The release gate names a "release approver" role; confirm the named human owner is recorded.